

```

#include "contiki.h"
#include "board-peripherals.h"
#include <stdio.h>
#include <stdint.h>
#include <sys/etimer.h>
PROCESS(hello_world_process, "hello_world process");
AUTOSTART_PROCESSES(&hello_world_process);
/* AUTOSTART_PROCESSES(&hello_world_process, &..., &..., ... ); s'il a plusieurs process */
PROCESS_THREAD(hello_world_process, ev, data)
{
    static struct etimer timer;
    PROCESS_BEGIN();
    etimer_set(&timer, CLOCK_SECOND * 10);
    while (1) {
        SENSOR_ACTIVITE(bmp_1000_sensor);
        /* SENSOR_ACTIVITE(bmp_280_sensor); pour l'humidité*/
        PROCESS_WAIT_EVENT();
        if (data == &hdc_1000_sensor) /* cet if est pour s'assurer que la data a bien reçu l'événement */
        {
            int temp = hdc_1000_sensor.value(HDC_1000_SENSOR_TYPE_TEMP);
            /* int temp = hdc_1000_sensor.value(HDC_1000_SENSOR_TYPE_HUMIDITY); pour l'humidité*/
            PROCESS_WAIT_UNTIL(etimer_expired(&timer));
            etimer_reset(&timer);
            printf("Temp: %d.%d\n", temp/100, temp%100);
        }
    }
    PROCESS_END();
}

```

dockerfile :

```

# Use an official Python runtime as a parent image
FROM python:2.7-slim

# Set the working directory to /app
WORKDIR /app

# Copy the current directory contents into the container at /app
COPY . /app

# Install any needed packages specified in requirements.txt
RUN pip install --trusted-host pypi.python.org -r requirements.txt

# Make port 80 available to the world outside this container
EXPOSE 80

# Define environment variable
ENV NAME World

# Run app.py when the container launches
CMD ["python", "app.py"]

```

dockerfile achraf :

```

FROM alpine:latest
RUN apk add gcc libc-dev

```

← exécute le script minimal alpine
 ↑ exécute la commande pour installer gcc

Core
 FP
 virtualisation
 Coms 2

```

app.py
hello python
print("Hello world")

```

```
COPY hw.c /achraf
WORKDIR /achraf/
RUN gcc /achraf/hw.c hw
CMD ./hw
```

compile
le fichier hw.c

achraf → home
hw.c → code.c
(2)

Pour assigner un thread à un domaine, il faut exécuter l'instruction suivante :

```
/* Set thread's domain */
seL4_Error seL4_DomainSet_Set(seL4_DomainSet_service,
seL4_Uint8 domain, seL4_TCB thread);
```

L'objectif de l'exercice est de créer un thread seL4 qui exécute une fonction passée en argument :

```
int new_thread(void *arg1, void *arg2, void *arg3) {
printf("Hello2: arg1 %p, arg2 %p, arg3 %p\n", arg1, arg2, arg3);
void (*func)(int) = arg1;
func(*(int *)arg2);
while(1);
}
```

Pour créer un nouveau thread, dans le fichier thread.c, un thread non typé est défini :

```
extern seL4_CPtr tcb_untyped;
```

Le thread à créer, nommé tcb_cap_slot dans le fichier, est un re-typage de ce thread. Pour retyper un thread non typé, la fonction seL4_Untyped_Retype est à exécuter. Sa signature est :

```
int seL4_Untyped_Retype(seL4_Untyped_service,
seL4_Word type,
seL4_Word size_bits,
seL4_CNode root,
seL4_Word node_index,
seL4_Word node_depth,
seL4_Word node_offset,
seL4_Wors num_objects);
```

Passons à la configuration du TCB. La fonction à utiliser est :

```
int seL4_TCB_Configure(seL4_TCB_service,
seL4_Word fault_ep,
seL4_CNode cspace_root,
seL4_Word cspace_root_data,
seL4_CPtr vspace_root,
seL4_Word vspace_root_data,
seL4_Word buffer,
seL4_CPtr bufferFrame);
```

Dans le répertoire camkes-mv-linux, exécutez la commande ninja.

Créez une interface tap :

```
ip tuntap add tap0 mode tap
ip addr add 10.0.120.100/24 dev tap0
ip link set dev tap0 up
```

TP

Flora
nuyon